

MIPS Instruction Set Overview

1. Arithmetic Instructions

Used for basic integer operations. All operands must be in registers.

- `add rd, rs, rt` // $rd = rs + rt$
- `sub rd, rs, rt` // $rd = rs - rt$
- `addi rt, rs, imm` // $rt = rs + \text{immediate}$

Example:

```
add $t0, $t1, $t2  # $t0 = $t1 + $t2
addi $t3, $t1, 10   # $t3 = $t1 + 10
```

2. Register Operands

MIPS uses 32 general-purpose registers:

- Temporary: `$t0 - $t9`
- Saved: `$s0 - $s7`
- Others: `$zero`, `$at`, `$v0-$v1`, `$a0-$a3`, `$k0-$k1`, `$sp`, `$fp`, `$ra`

Note: `$zero` always holds the constant value 0.

3. Logical Operations

Perform bitwise logic on register contents.

- `and rd, rs, rt` // $rd = rs \text{ AND } rt$
- `or rd, rs, rt` // $rd = rs \text{ OR } rt$
- `xor rd, rs, rt` // $rd = rs \text{ XOR } rt$
- `nor rd, rs, rt` // $rd = \text{NOT}(rs \text{ OR } rt)$

- `andi rt, rs, imm` // rt = rs AND immediate
- `ori rt, rs, imm` // rt = rs OR immediate

Example:

```
and $t0, $t1, $t2  # $t0 = $t1 & $t2
ori $t3, $t1, 15   # $t3 = $t1 | 15
```

4. Shift Operations

Shift bits left or right. Useful for bit manipulation and multiplication/division by powers of two.

- `sll rd, rt, shamt` // Shift left logical
- `srl rd, rt, shamt` // Shift right logical
- `sra rd, rt, shamt` // Shift right arithmetic

Example:

```
sll $t0, $t1, 2    # $t0 = $t1 << 2
sra $t2, $t3, 1    # $t2 = $t3 >> 1 (signed)
```

5. Immediate Operands

Used when one operand is a constant.

- Must be a 16-bit signed value.
- Common instructions: `addi`, `andi`, `ori`, `slti`

Example:

```
slti $t0, $s1, 10  # $t0 = ($s1 < 10) ? 1 : 0
```